

Salesforce Developer Bridge Program

Day 6 Documentation

Apex Triggers and Trigger Handler / Service Architecture

Field	Details
Student Name	Sudharshan Kuppili
PIN No.	24PA5A0517
Program	Salesforce Developer Bridge Program
Day	Day 6
Project	Placement Management System
Technology	Apex, SOQL, DML, Apex Triggers, Trigger Handler / Service
Main Topic	Apex Triggers and Trigger Handler / Service

Table of Contents

3. Day 6 Objective.....	3
4. Business Scenario.....	3
5. Apex Trigger Overview.....	3
6. Trigger Architecture.....	4
7. Before Insert — Duplicate Application Prevention.....	5
8. Bulkification and Governor Limits.....	7
9. addError() and Business Validation.....	8
10. After Update — Application Status Change.....	8
11. Related Student Record Update.....	11
12. ApplicationService / Trigger Handler Architecture.....	12
13. Testing and Debugging.....	13
14. Test Results Summary.....	15
15. Debugging / Engineering Thinking.....	16
16. Interview Corner.....	16
17. Sprint Retrospective.....	17
18. Key Learnings.....	17
19. Conclusion.....	18
20. Screenshot / Evidence Appendix.....	18

3. Day 6 Objective

The objective of Day 6 was to understand and implement Apex Triggers in the Placement Management System.

The day's work focused on the following key areas:

- Understand Before Insert and After Update Trigger events.
- Use Trigger context variables to identify the current database operation.
- Prevent duplicate Placement Applications for the same Student and Job.
- Use `addError()` to stop invalid records from being saved.
- Build bulk-safe Apex logic that avoids SOQL and DML inside loops.
- Separate Trigger logic from business logic using an `ApplicationService` class.
- Update a related Student record automatically when an application becomes Selected.

4. Business Scenario

In the Placement Management System, a student should not be able to submit the same job application more than once. Allowing duplicate applications would create confusion for recruiters and distort placement reporting, so this rule needed to be enforced automatically at the database level rather than relying on manual checks.

The system also needs to react when an application status changes. When an application changes from Shortlisted to Selected, the related Student record should be updated to show the student's placement status as Selected. This ensures that the Placement Officer always sees an accurate, up-to-date picture of each student's placement outcome without needing to update multiple records by hand.

Both requirements call for automatic, event-driven behaviour — exactly the kind of responsibility that Apex Triggers, working together with a dedicated service class, are designed to handle.

5. Apex Trigger Overview

An Apex Trigger is Apex code that executes automatically before or after specific database events on Salesforce records. Rather than waiting for a user to click a button, a Trigger allows the Placement Management System to observe important events — such as a new application being created or an existing application being updated — and respond to them automatically.

5.1 Trigger Events Used in This Project

The Placement Application Trigger handles two events:

- Before Insert — used for duplicate validation before a new application is saved.
- After Update — used to detect a status change and update the related Student record.

5.2 Trigger Context Variables

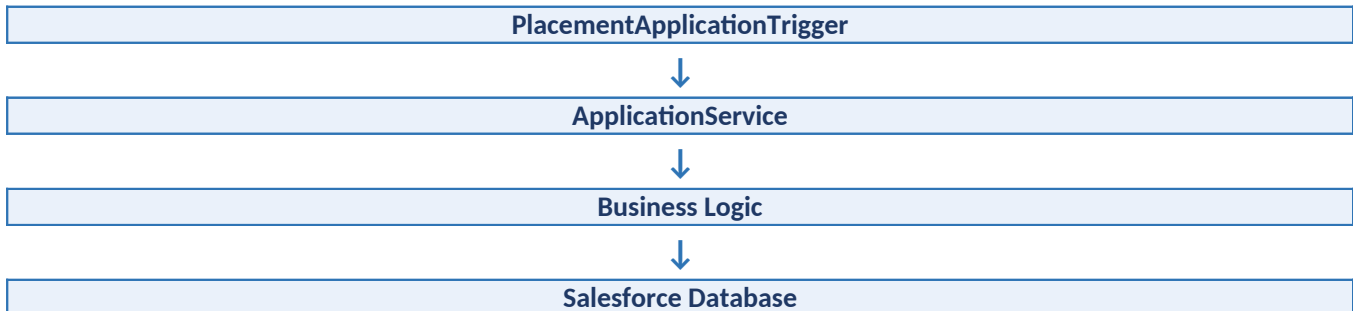
The following Trigger context variables were used to implement this functionality:

- `Trigger.new` — the list of new/updated record versions in the current transaction.
- `Trigger.oldMap` — a map of the previous version of each record, keyed by Id.

- `Trigger.isBefore` — true when the Trigger is executing before the record is saved.
- `Trigger.isAfter` — true when the Trigger is executing after the record is saved.
- `Trigger.isInsert` — true when the triggering operation is an insert.
- `Trigger.isUpdate` — true when the triggering operation is an update.

6. Trigger Architecture

The Trigger delegates all business logic to the `ApplicationService` class instead of containing that logic directly. This keeps the Trigger itself short, readable, and easy to maintain.



Because the Trigger only recognises the event and calls the correct `ApplicationService` method, it remains lightweight. All of the actual decision-making — querying records, validating business rules, and updating related data — is centralised inside `ApplicationService`, which makes the logic easier to test, reuse, and extend.

6.1 PlacementApplicationTrigger — Source Code

```

trigger PlacementApplicationTrigger
on Placement_Application__c (before insert, after update) {

    if (Trigger.isBefore && Trigger.isInsert) {
        ApplicationService.validateNewApplications(Trigger.new);
    }

    if (Trigger.isAfter && Trigger.isUpdate) {
        ApplicationService.handleApplicationUpdate(
            Trigger.new,
            Trigger.oldMap
        );
    }
}
  
```

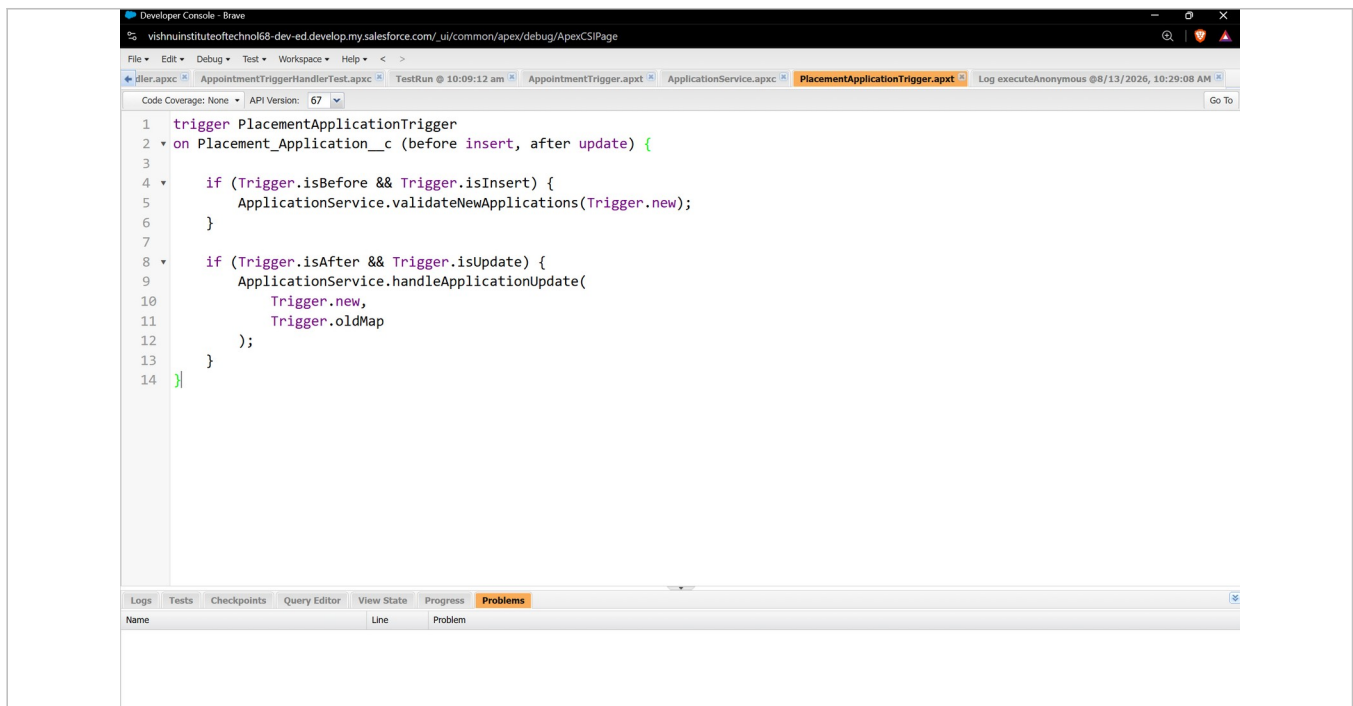


Figure 1: PlacementApplicationTrigger source code showing the Before Insert and After Update blocks that delegate to ApplicationService.

7. Before Insert — Duplicate Application Prevention

7.1 Business Requirement

A student should not be allowed to apply for the same job more than once. The duplicate combination being checked is Student + Job.

7.2 Implementation Approach

- Collect the Student and Job Ids from the incoming (Trigger.new) records.
- Run a single SOQL query to retrieve existing applications for those Students and Jobs.
- Build Student + Job combinations from the existing records.
- Use addError() to block any new application that matches an existing combination.

7.3 ApplicationService — validateNewApplications()

The relevant portion of the ApplicationService class collects Student and Job Ids in Sets before performing the duplicate check, which keeps the logic bulk-safe:

```

public static void validateNewApplications(
    List<Placement_Application__c> newApplications
) {
    Set<Id> studentIds = new Set<Id>();
    Set<Id> jobIds = new Set<Id>();

    for (Placement_Application__c application : newApplications) {

```

```

    if (application.Student__c != null) {
        studentIds.add(application.Student__c);
    }

    if (application.Job__c != null) {
        jobIds.add(application.Job__c);
    }
}
// Existing applications for these Student + Job combinations are then
// queried in a single SOQL call, and addError() is used to block
// any new record that matches an existing combination.

```

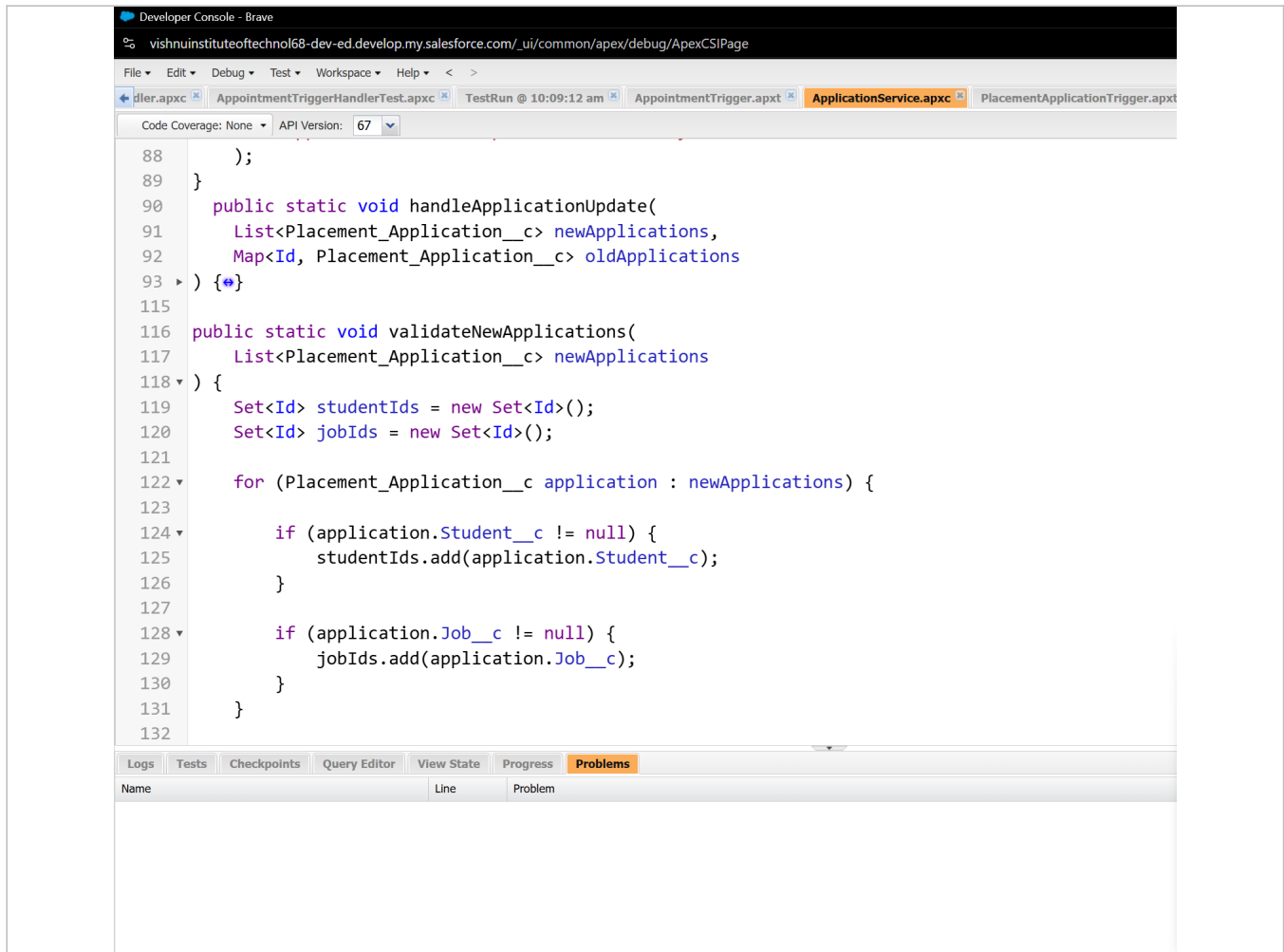


Figure 2: ApplicationService.apxc — validateNewApplications() collecting Student and Job Ids in Sets ahead of the duplicate check.

7.4 Duplicate Test — Apex Code Executed

To verify the rule, a second Placement Application record was created in Execute Anonymous using the same Student and Job combination as an existing record:

```

Placement_Application__c duplicateApplication =
    new Placement_Application__c(
        Student__c = 'a08NS00001UpMRcYAN',

```

```

        Job__c = 'a09NS00000YRzdgYAD',
        Application_Date__c = Date.today(),
        Status__c = 'Applied'
    );

insert duplicateApplication;

```

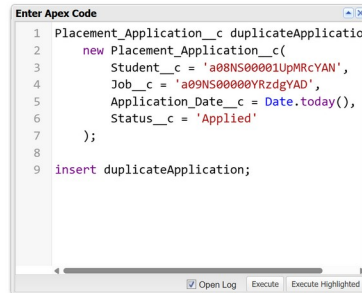


Figure 3: Execute Anonymous window used to attempt insertion of a duplicate Placement Application for the same Student and Job.

7.5 Result

The duplicate application insert was blocked with the following meaningful error message:

“Duplicate application found. This student has already applied for this job.”

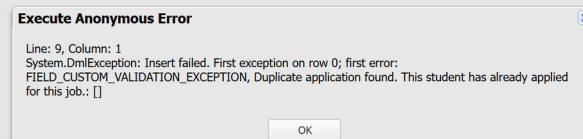
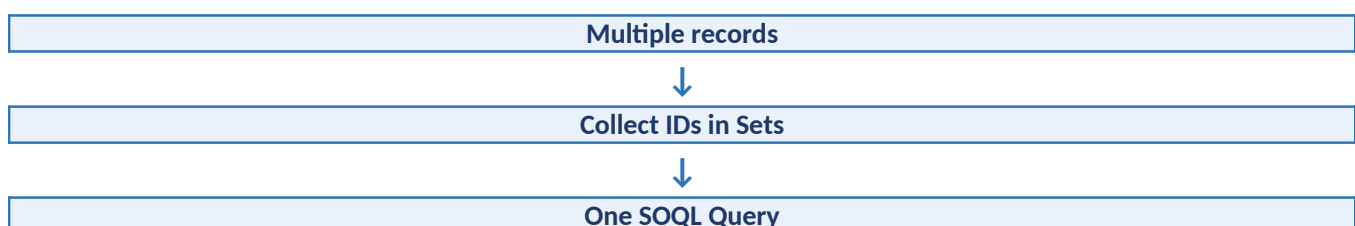


Figure 4: Execute Anonymous Error dialog confirming the duplicate application was blocked by addError().

8. Bulkification and Governor Limits

The Trigger logic was designed to avoid SOQL and DML operations inside loops. Salesforce enforces Governor Limits on the number of SOQL queries and DML statements that can be executed within a single transaction, so placing either operation inside a loop can quickly exceed those limits when many records are processed together.

8.1 Approach





As shown in Figure 2, Student and Job Ids from every incoming record are first collected into Set<Id> variables. A single SOQL query is then used to retrieve all related existing applications at once, and the resulting records are processed in memory before any DML statement is issued. This same bulk-safe pattern — collect, query once, process in memory — is also used in the After Update logic (Section 10).

8.2 Learning Outcome

This exercise reinforced why SOQL and DML should never be placed inside loops, and how collections such as Set and List can be used to build Apex code that behaves correctly whether one record or many records are being processed in the same transaction.

9. addError() and Business Validation

The Trigger uses `addError()` to prevent invalid duplicate applications from being saved.

9.1 Benefits

- Stops the invalid record from being inserted.
- Provides a meaningful message to the user.
- Enforces the business rule during the database transaction, before any data is committed.

9.2 Test Result

As shown earlier, the duplicate insert failed as expected, with Salesforce raising a `FIELD_CUSTOM_VALIDATION_EXCEPTION` carrying the custom message defined by `addError()`:

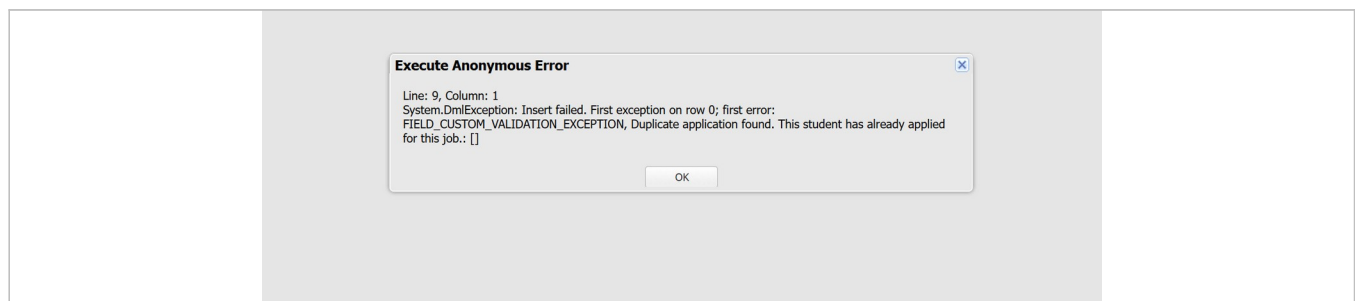
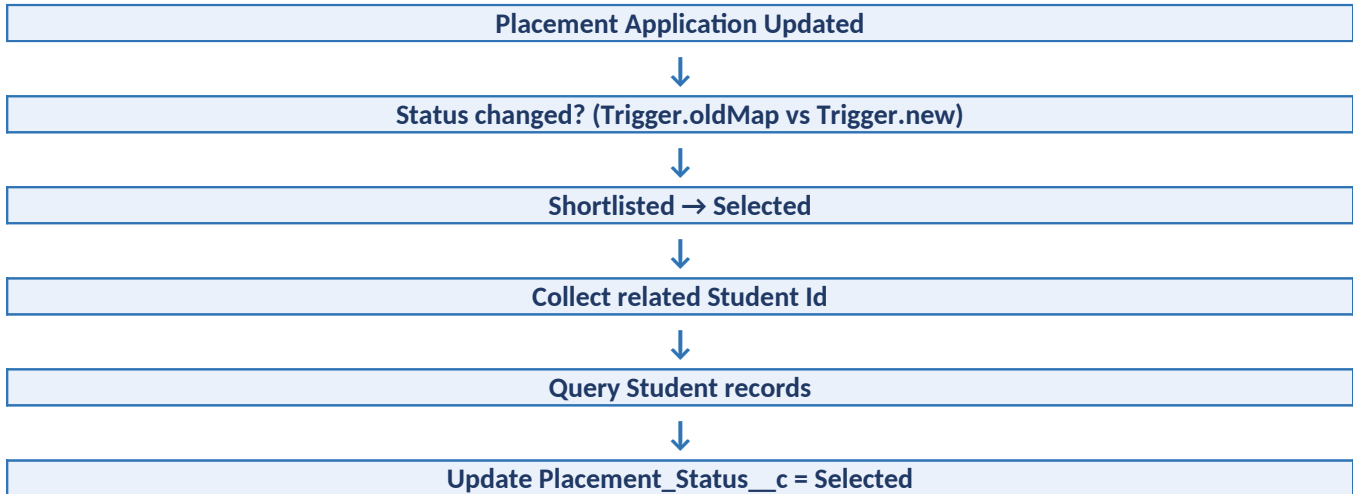


Figure 5: Duplicate application prevented using `addError()` (repeated from Figure 4 for reference in this section).

10. After Update — Application Status Change

The After Update logic checks whether `Status__c` changed, and specifically watches for the important business transition from Shortlisted to Selected. The comparison is made by looking up each record's previous version through `Trigger.oldMap`, keyed by the record's Id.

10.1 Processing Flow



10.2 ApplicationService — handleApplicationUpdate()

The After Update handler loops through Trigger.new, compares each record against its corresponding entry in Trigger.oldMap, and collects the Ids of Students whose application has just become Selected:

```

public static void handleApplicationUpdate(
    List<Placement_Application__c> newApplications,
    Map<Id, Placement_Application__c> oldApplications
) {
    Set<Id> studentIds = new Set<Id>();

    for (Placement_Application__c newApplication : newApplications) {
        Placement_Application__c oldApplication =
            oldApplications.get(newApplication.Id);

        if (
            oldApplication.Status__c != newApplication.Status__c &&
            newApplication.Status__c == 'Selected' &&
            newApplication.Student__c != null
        ) {
            studentIds.add(newApplication.Student__c);
        }
    }

    if (studentIds.isEmpty()) {
        return;
    }

    List<Student__c> students = [
        SELECT Id, Placement_Status__c
        FROM Student__c
        WHERE Id IN :studentIds
    ];

    for (Student__c student : students) {
        student.Placement_Status__c = 'Selected';
    }
}
  
```

```
// Students are then updated in a single bulk DML statement.
```

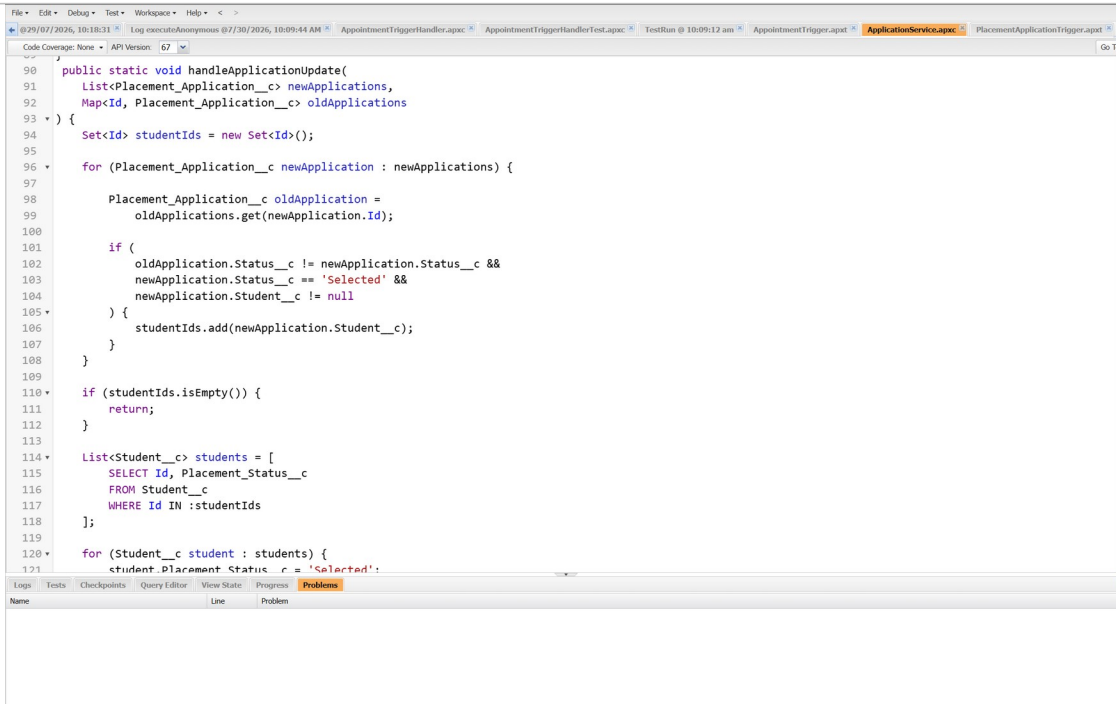


Figure 6: ApplicationService.apxc — handleApplicationUpdate() comparing Trigger.oldMap against Trigger.new and collecting Student Ids for the Shortlisted → Selected transition.

10.3 Status Change Test

The status update was exercised directly through ApplicationService.updateApplicationStatus() in Execute Anonymous, with the PlacementApplicationTrigger open alongside to confirm the trigger fired correctly:

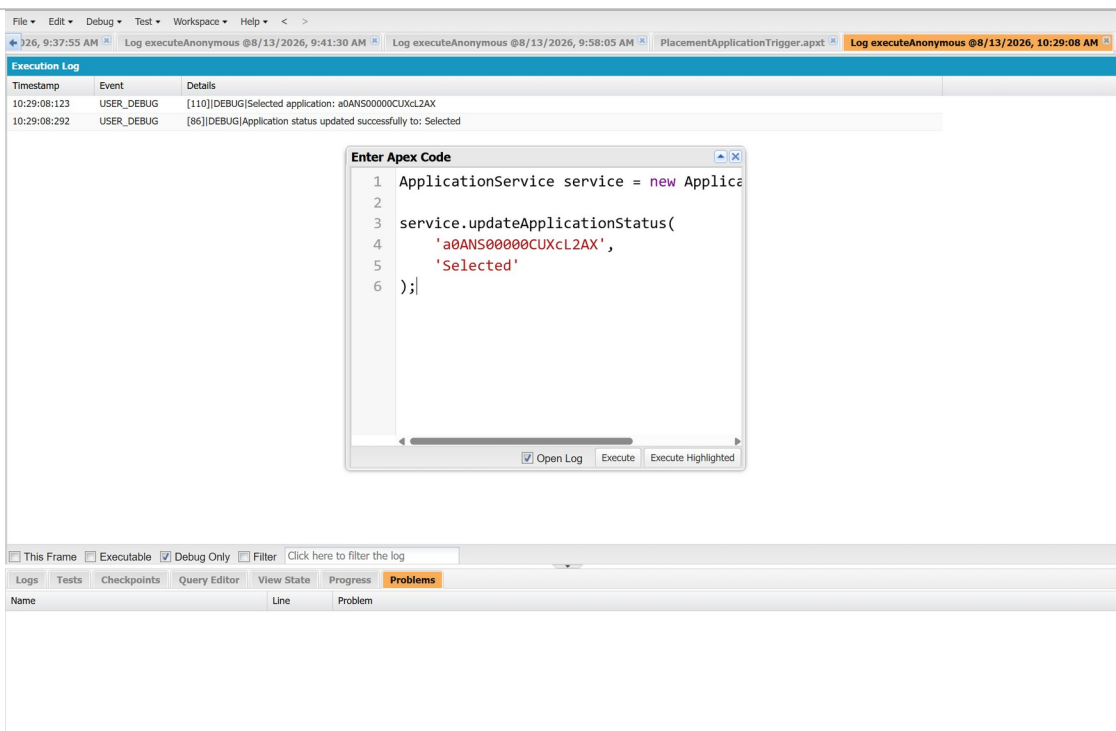


Figure 7: Execute Anonymous log showing the Placement Application status being updated to Selected, with the PlacementApplicationTrigger open for reference.

11. Related Student Record Update

A Placement_Status__c picklist field was created on Student__c to support the related-record update described above.

When the Placement Application status changes to Selected, the related Student record is updated:

```
Student.Placement_Status__c = 'Selected';
```

11.1 Test Result

The application was changed from Shortlisted to Selected, and the debug log confirmed both that the correct application was selected and that its status was updated successfully:

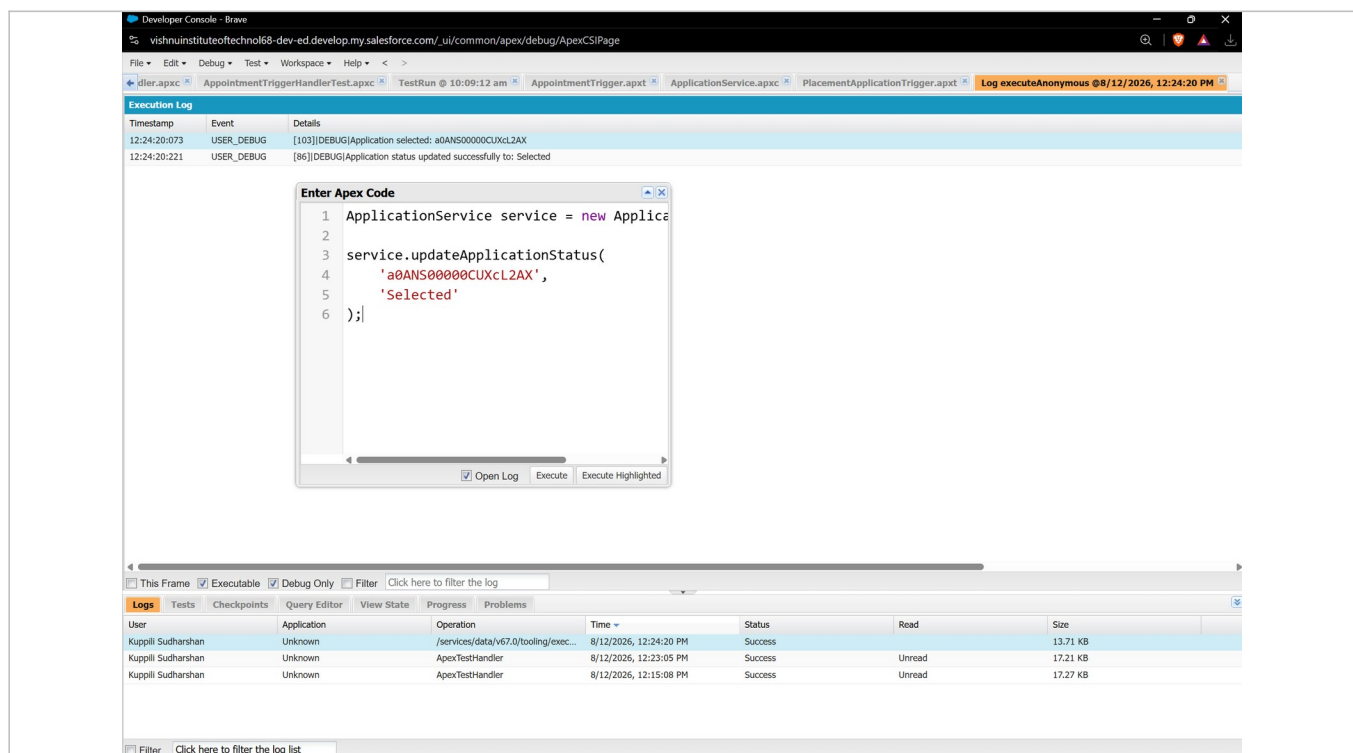


Figure 8: Execute Anonymous debug output confirming “Application selected” and “Application status updated successfully to: Selected”.

A follow-up query against the Placement Application confirmed the Selected status together with the related Student's name, verifying that the correct Student record was linked to the update:

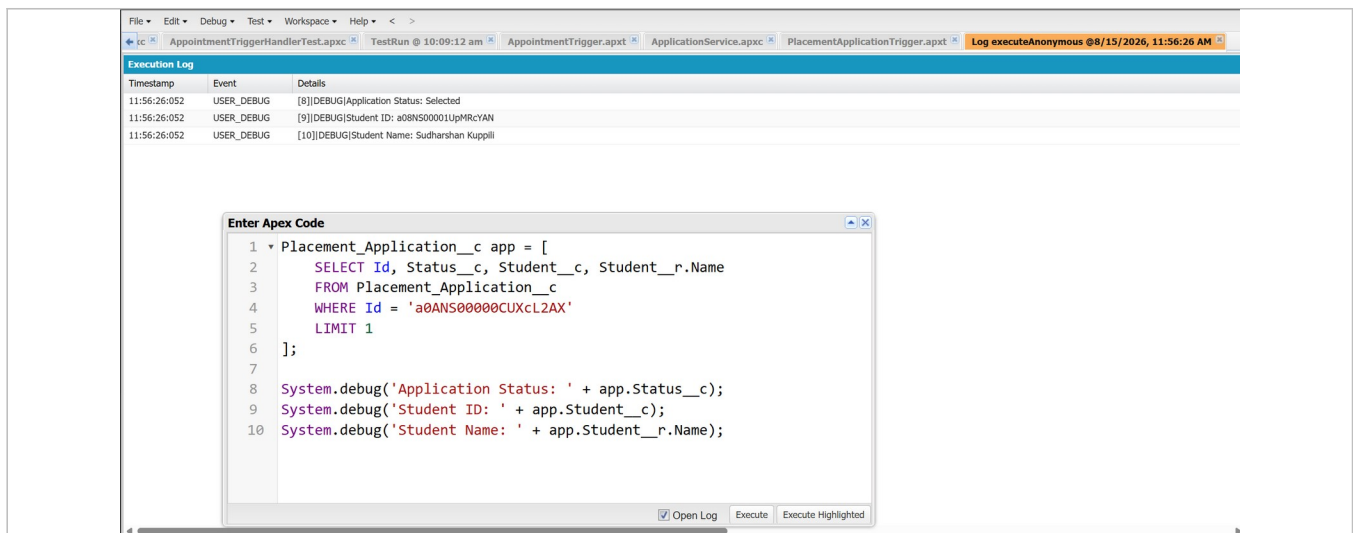


Figure 9: Execute Anonymous debug output verifying Application Status: Selected, Student ID, and Student Name for the updated application.

12. ApplicationService / Trigger Handler Architecture

The Trigger delegates business logic to ApplicationService instead of containing all logic directly inside the Trigger. This separation improves maintainability, reusability, testability, readability, and bulk processing behaviour.

12.1 Service Methods

- `validateNewApplications()` — runs the Before Insert duplicate-prevention logic (Figure 2).
- `handleApplicationUpdate()` — runs the After Update status-change logic (Figure 6).
- `submitApplication()` — used to create new Placement Application records.
- `updateApplicationStatus()` — used to update the `Status__c` field on an existing application.

12.2 `updateApplicationStatus()` — Source Code

This method was used directly in Execute Anonymous throughout Day 6 testing to move applications between statuses (see Figures 7 and 8):

```

public void updateApplicationStatus(Id applicationId, String newStatus) {

    Placement_Application__c application = [
        SELECT Id, Status__c, Interview_Date__c
        FROM Placement_Application__c
        WHERE Id = :applicationId
        LIMIT 1
    ];

    application.Status__c = newStatus;

    if (newStatus == 'Interview Scheduled') {
        application.Interview_Date__c = Date.today();
    }
}

```

```

update application;

System.debug(
    'Application status updated successfully to: ' + newStatus
);
}

```

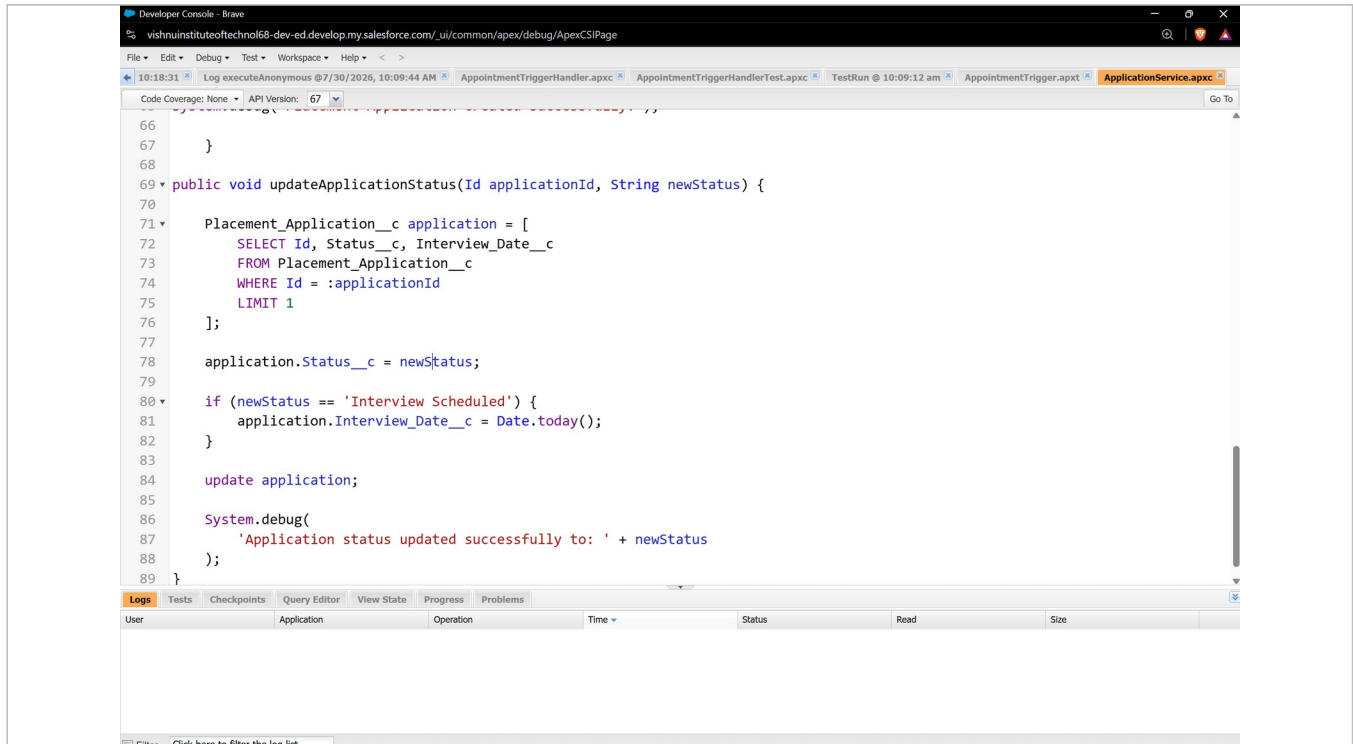


Figure 10: ApplicationService.apxc — updateApplicationStatus(), the shared method used to move an application between statuses during testing.

13. Testing and Debugging

Test 1 — Duplicate Application

Objective: Confirm that a duplicate Placement Application (same Student + same Job) is blocked.

Steps: Inserted a new Placement_Application__c record in Execute Anonymous using the Student__c and Job__c values of an existing application (Figure 3).

Expected Result: Duplicate application should be blocked.

Actual Result: Insert failed with FIELD_CUSTOM_VALIDATION_EXCEPTION and the message “Duplicate application found. This student has already applied for this job.” (Figure 4).

Status: PASSED

Test 2 — Duplicate Record Verification

Objective: Confirm that only the original application record exists for the Student + Job combination after the duplicate insert was blocked.

Steps: Queried Placement_Application__c WHERE Student__c and Job__c match the tested combination and printed the result count.

Expected Result: Only the original application should remain.

Actual Result: Application count confirmed as 1.

Status: PASSED

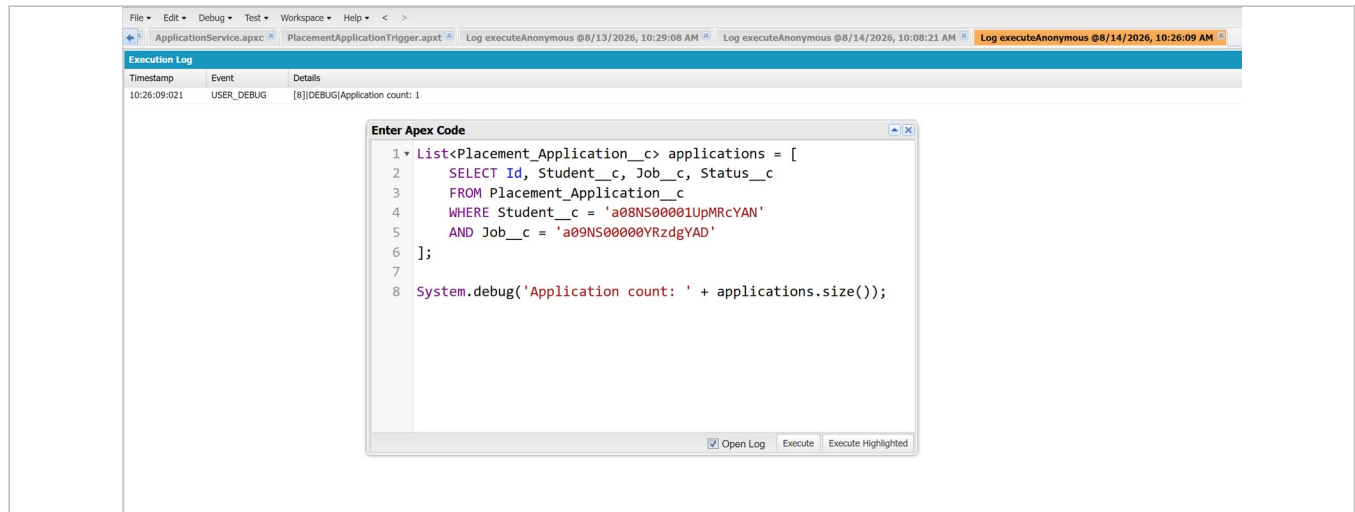


Figure 11: Execute Anonymous debug output confirming Application count: 1 after the duplicate insert was blocked.

Test 3 — After Update Cascade (Shortlisted → Selected)

Objective: Confirm that when an Application Status changes from Shortlisted to Selected, the related Student's Placement Status is updated to Selected.

Steps: The application was first set to Shortlisted using `updateApplicationStatus()`, then updated to Selected in a subsequent Execute Anonymous call while the PlacementApplicationTrigger was open (Figure 7).

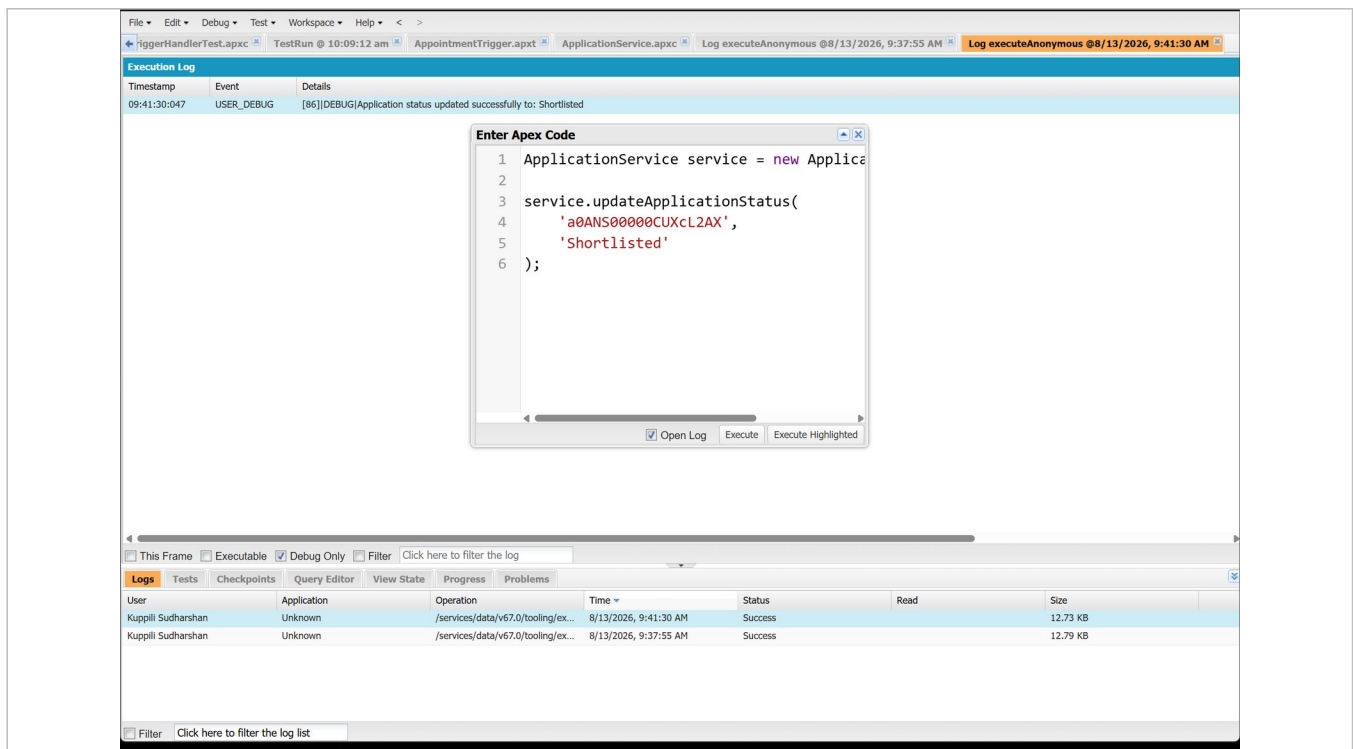


Figure 12: Execute Anonymous debug output showing the application status being set to Shortlisted, ahead of the Selected transition test.

Expected Result: When Application Status changes from Shortlisted to Selected, the related Student Placement Status should become Selected.

Actual Result: The debug log confirmed “Application selected” and “Application status updated successfully to: Selected” (Figure 8), and a follow-up query confirmed Application Status: Selected together with the correct Student Name (Figure 9).

Status: PASSED

Test 4 — Trigger Execution and Service Compilation

Objective: Confirm the Trigger and ApplicationService executed without unexpected errors across all Execute Anonymous runs.

Expected Result: No unexpected errors; service class compiles successfully.

Actual Result: All Execute Anonymous runs completed successfully, and no compile errors were reported in the Developer Console Problems panel (Figures 6, 7 and 9).

Status: PASSED

14. Test Results Summary

Test	Expected Result	Actual Result	Status
Duplicate application	Insert blocked	Duplicate blocked	PASSED
Duplicate count	Only 1 application	Count = 1	PASSED

Test	Expected Result	Actual Result	Status
Shortlisted → Selected	Student updated	Placement Status = Selected	PASSED
Trigger execution	No unexpected errors	Successful	PASSED
Service class	Compiles successfully	No errors	PASSED

15. Debugging / Engineering Thinking

Key lessons from Day 6:

- Before Triggers are useful for validation and modifying records before database save.
- After Triggers are useful when actions depend on the saved record and its relationships.
- SOQL and DML should not be placed inside loops.
- Sets and Lists help build bulk-safe Apex logic.
- `addError()` can prevent invalid data from being saved.
- Trigger context variables provide information about the current database operation.
- Business logic should be separated from the Trigger using a service or handler class.
- Business validation should happen before DML whenever possible.
- Related-record updates should be implemented in a bulk-safe manner.

16. Interview Corner

Q1. What is an Apex Trigger?

An Apex Trigger is Apex code that executes automatically before or after specific database events on Salesforce records.

Q2. What is the difference between Before and After Triggers?

Before Triggers are commonly used for validation and modifying the record before it is saved. After Triggers are commonly used for actions that depend on the saved record or related records.

Q3. Why use `Trigger.oldMap`?

`Trigger.oldMap` provides the previous version of records by Id, allowing us to compare old and new field values during update operations.

Q4. What is `Trigger.new`?

`Trigger.new` is a list containing the new versions of the records that are currently being inserted or updated in the transaction.

Q5. Why should we avoid SOQL inside loops?

Salesforce has Governor Limits. SOQL inside loops can quickly exceed the allowed number of queries when many records are processed.

Q6. What is `addError()`?

`addError()` prevents a record from being saved and displays a meaningful validation error to the user.

Q7. What is bulkification?

Bulkification is the practice of writing Apex code that performs correctly and efficiently whether it processes one record or thousands, typically by collecting data into Sets/Lists, running one SOQL query, and performing one DML operation instead of many.

Q8. Why keep Trigger logic separate from business logic?

A lightweight Trigger is easier to maintain and test. Business logic can be centralized in a reusable service or handler class, such as `ApplicationService`.

Q9. How does the After Update logic update a related record?

It collects the Ids of Students whose application just became Selected in a Set, queries all required Student records in a single SOQL query, updates them in memory, and performs one DML operation.

17. Sprint Retrospective

What worked well?

The duplicate prevention and After Update related-record update worked successfully. The Trigger and `ApplicationService` worked together to enforce the business rules.

Which part felt most natural?

Understanding Trigger context variables and separating the Trigger from the service logic became clearer through hands-on testing.

Which part was challenging?

Understanding when to use Before versus After Trigger events and ensuring that the After Update logic only runs when the status actually changes required careful reasoning.

What would be improved?

The service class could be further divided into smaller reusable methods and could be supported with comprehensive Apex test classes for different business scenarios.

How would the system support a large number of students?

The system should use bulkified Apex, efficient SOQL, Sets/Lists/Maps, avoid SOQL and DML inside loops, and use reusable services that respect Governor Limits.

18. Key Learnings

By completing Day 6, the following was learned:

- How Apex Triggers respond to database events.
- How Before Insert can enforce duplicate-prevention rules.
- How After Update can respond to changes in record status.
- How Trigger.new and Trigger.oldMap are used together.
- How addError() prevents invalid records.
- How to write bulk-safe SOQL and DML logic.
- Why Trigger logic should be separated from business logic.
- How to update a related record safely after a business event.

19. Conclusion

Day 6 strengthened the understanding of Apex Triggers and Trigger-Service architecture in Salesforce.

The Placement Management System now demonstrates two important business automation scenarios: preventing duplicate job applications before insertion, and updating a related Student record when an application becomes Selected.

The implementation also follows important Salesforce development practices such as bulkification, efficient SOQL, controlled DML, meaningful validation errors, Trigger context handling, and separation of business logic.

20. Screenshot / Evidence Appendix

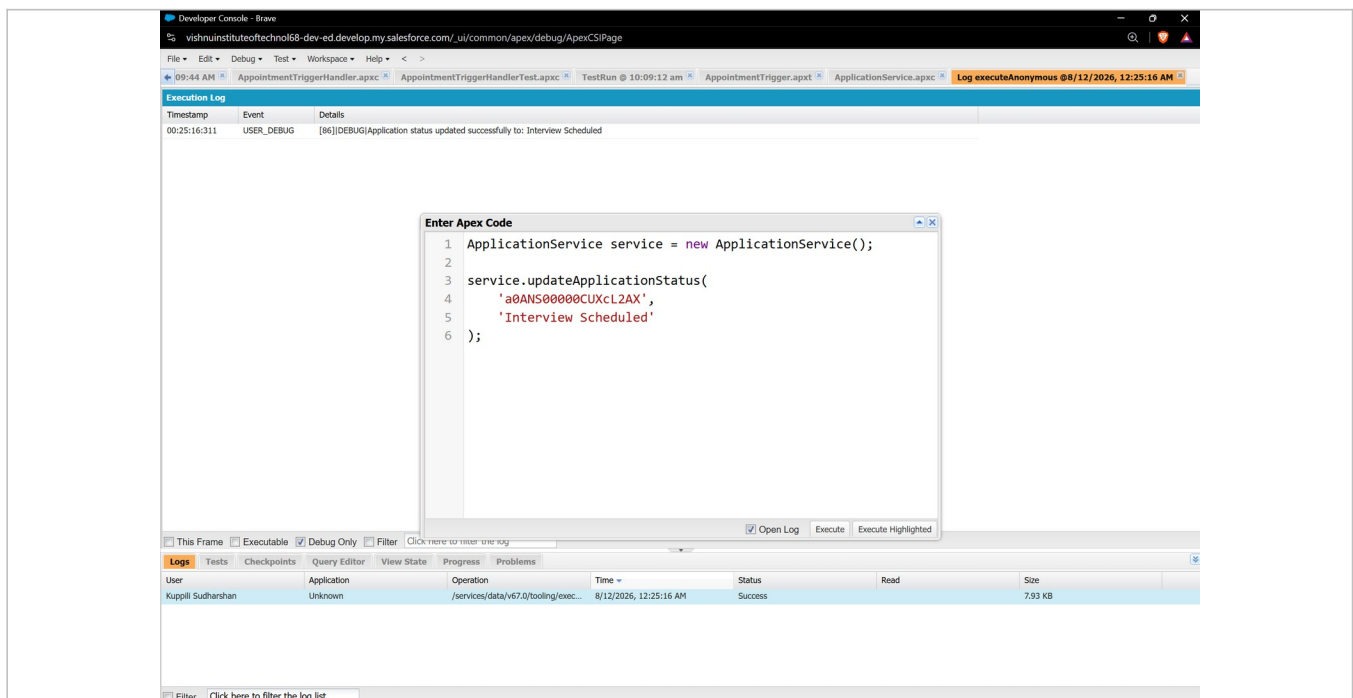
The table below lists every screenshot captured during Day 6 work, in the order it appears in this document, along with what each screenshot proves.

Fig.	Section	What It Proves
1	Trigger Architecture	PlacementApplicationTrigger source code (Before Insert / After Update).
2	Before Insert	ApplicationService.validateNewApplications() collecting Student/Job Ids.
3	Before Insert	Apex code used to attempt a duplicate application insert.
4 / 5	Before Insert / addError()	Execute Anonymous error confirming the duplicate was blocked.
6	After Update	ApplicationService.handleApplicationUpdate() Shortlisted → Selected logic.
7	After Update	Status update to Selected executed with the Trigger tab open.
8	Related Student Update	Debug log confirming the application was selected and updated.
9	Related Student Update	Verification query showing Status: Selected and Student Name.
10	ApplicationService Architecture	updateApplicationStatus() source code.

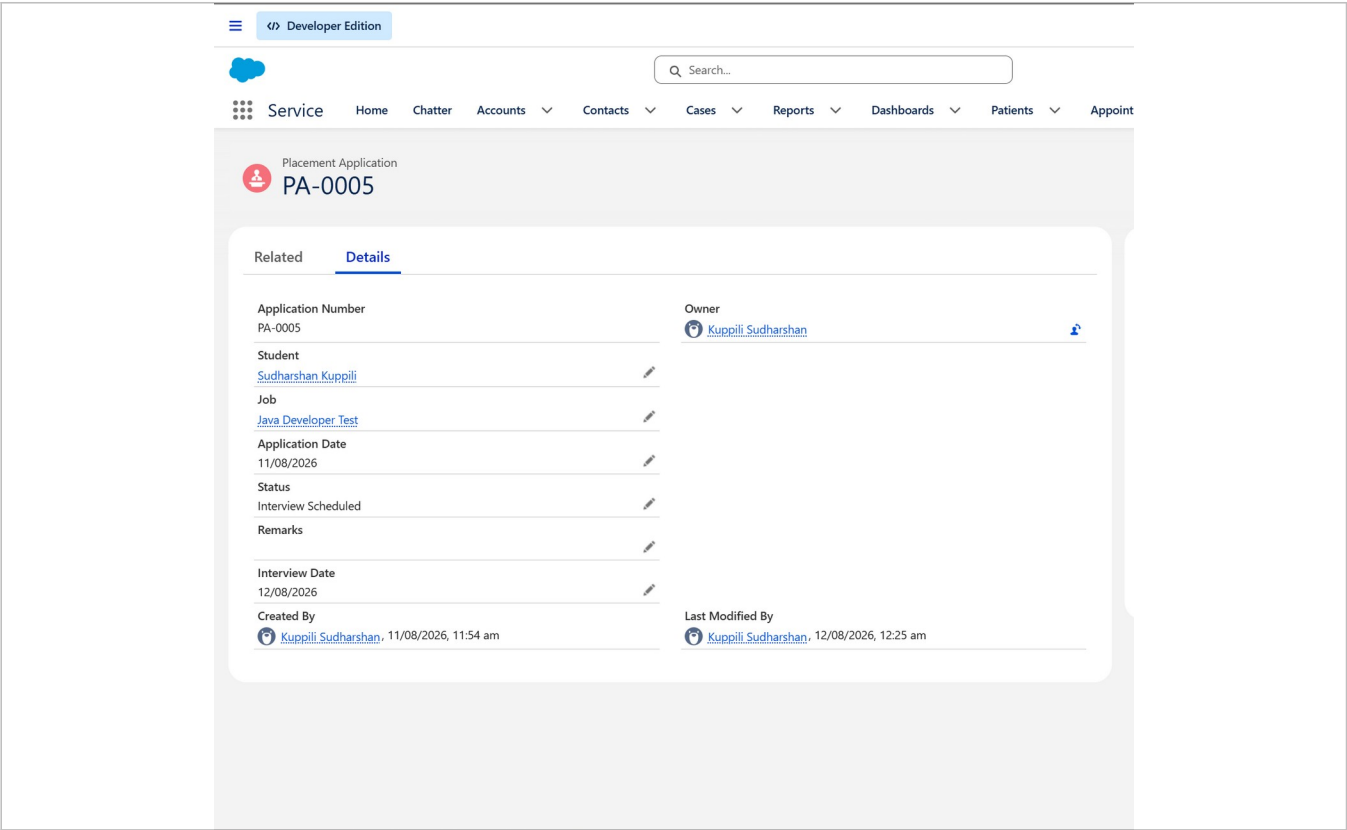
Fig.	Section	What It Proves
11	Testing	Application count query confirming only 1 record exists (no duplicate).
12	Testing	Application status set to Shortlisted ahead of the Selected test.

Supplementary Screenshots

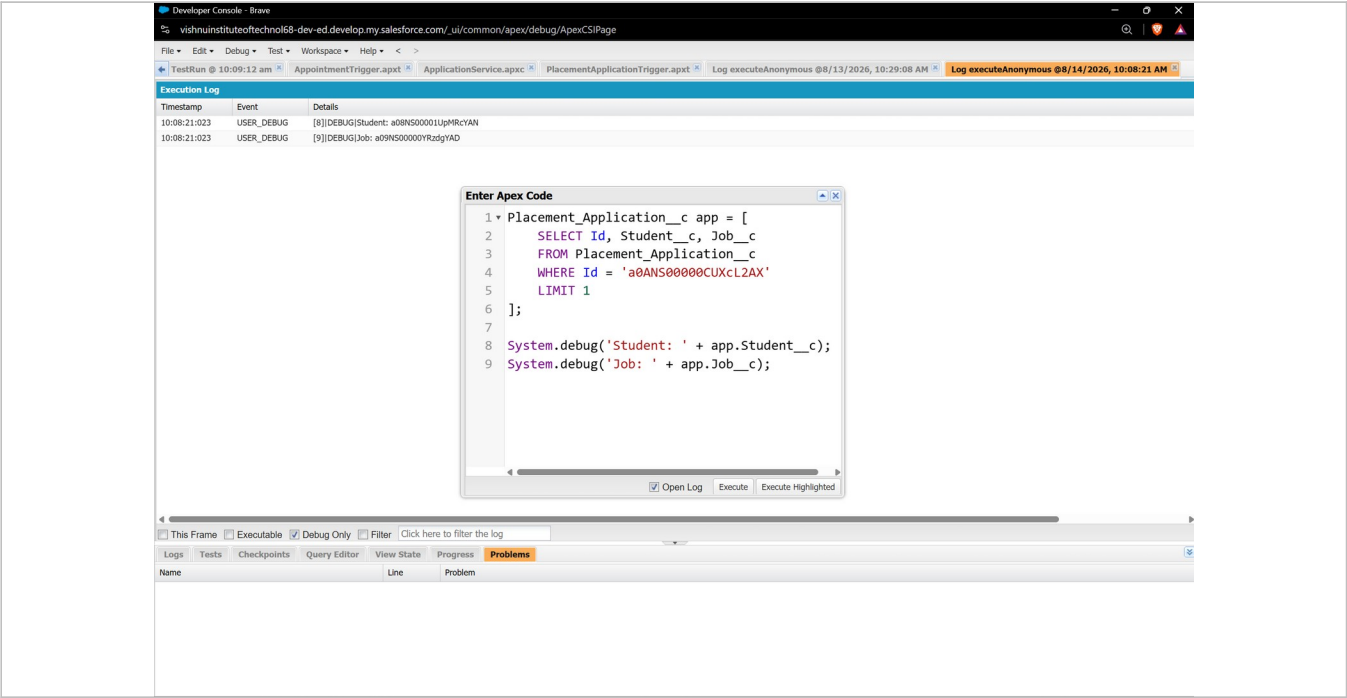
The following screenshots were captured during Day 6 work but could not be confidently mapped to a single required section above. They are included here as additional evidence rather than being forced into a section.



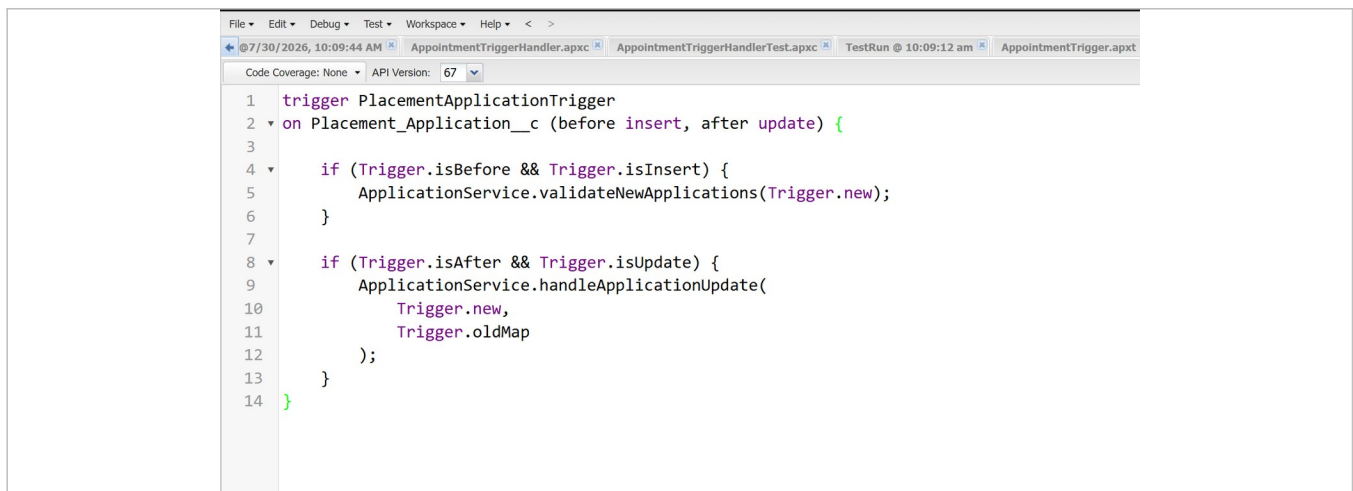
Supplementary Figure A: Execute Anonymous log showing `updateApplicationStatus()` used to set an application's status to Interview Scheduled.



Supplementary Figure B: Placement Application record PA-0005 showing Status = Interview Scheduled, corresponding to Supplementary Figure A.



Supplementary Figure C: Execute Anonymous debug output verifying the Student and Job Ids on a queried Placement Application record.



The screenshot shows a Salesforce Developer Console window with the following details:

- Menu Bar:** File, Edit, Debug, Test, Workspace, Help.
- Tab Bar:** @7/30/2026, 10:09:44 AM, AppointmentTriggerHandler.apxc, AppointmentTriggerHandlerTest.apxc, TestRun @ 10:09:12 am, AppointmentTrigger.apxt.
- Code Coverage:** None. **API Version:** 67.
- Source Code:**

```
1 trigger PlacementApplicationTrigger
2 on Placement_Application__c (before insert, after update) {
3
4     if (Trigger.isBefore && Trigger.isInsert) {
5         ApplicationService.validateNewApplications(Trigger.new);
6     }
7
8     if (Trigger.isAfter && Trigger.isUpdate) {
9         ApplicationService.handleApplicationUpdate(
10             Trigger.new,
11             Trigger.oldMap
12         );
13     }
14 }
```

Supplementary Figure D: PlacementApplicationTrigger source code viewed in a separate Developer Console window (duplicate view of Figure 1).